

Oznacz jako wykonane

Cel ćwiczenia

Celem ćwiczenia jest przygotowanie programu równoległego przybliżającego liczbę π .

Wprowadzenie

W ramach tego laboratorium będziemy dążyć do implementacji, a następnie analizy efektywności programu równoległego. Analiza taka daje bardzo intuicyjne i łatwe do interpretacji wyniki w przypadku problemów *naturalnie równoległych*. Jest to podzbiór problemów, w których część niemożliwa do zrównoleglenia jest pomijalna i nie ma wpływu na czas wykonania.

Przykładem problemu naturalnie równoległego jest przybliżanie liczby π metodą Monte Carlo (MC). Metoda ta jest stosowana do modelowania matematycznych procesów zbyt złożonych (obliczanie całek, łańcuchy procesów statystycznych), aby można było przewidzieć ich wyniki za pomocą podejścia analitycznego. Istotną rolę w metodzie MC odgrywa losowanie.

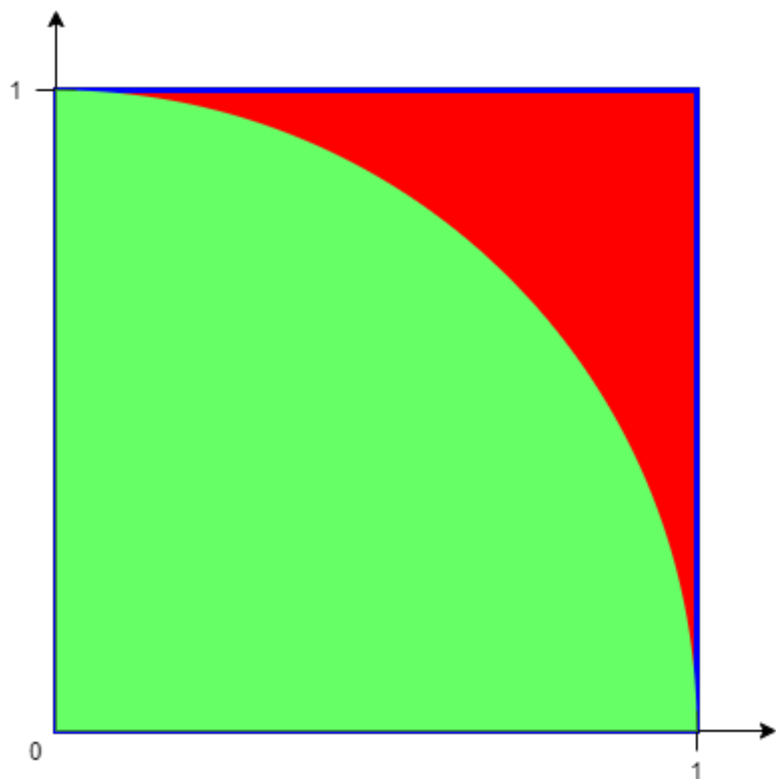
Algorytm obliczania liczby π metodą MC korzysta z zależności pomiędzy wartością liczby π a stosunkiem pola powierzchni koła o promieniu r do pola powierzchni kwadratu opisanego na tym kole:

- pole kwadratu: $P_s = (2r)^2 = 4r^2$,
- pole koła: $P_c = \pi r^2$,
- stosunek pola koła do pola kwadratu: $\frac{P_c}{P_s} = \frac{\pi r^2}{4r^2} = \frac{\pi}{4}$,
- finalnie: $\pi = 4 \cdot \frac{P_c}{P_s}$.

W efekcie, obliczenie liczby π metodą MC wymaga wykonania następujących kroków:

1. Wylosowanie pewnej liczby punktów rozłożonych równomiernie w kwadracie o boku $2r$ - przybliżamy w ten sposób pole kwadratu.
2. Zliczenie tych z punktów, które znajdują się wewnątrz koła wpisanego w ten kwadrat - przybliżamy w ten sposób pole koła.
3. Podzielić drugą wartość przez pierwszą, a wynik pomnożyć przez 4 - przybliżamy w ten sposób π .

Dla ułatwienia, problem można uprościć do jednej ćwiartki koła i kwadratu, a jako promień przyjąć wartość 1. Losujemy zatem punkty równomiernie rozłożone w niebieskim kwadracie (P_s), a dodatkowo zliczamy te leżące w zielonym wycinku koła (P_c) (patrz rysunek poniżej).



Zrównoleglenie tego algorytmu jest naturalne (jak sugeruje nazwa *naturalna równoległość*). Parametrem, który dyktuje rozmiar problemu, jest liczba punktów do wylosowania – im więcej punktów, tym lepsze przybliżenie. Podział pracy pomiędzy węzły obliczeniowe sprowadza się więc do podziału liczby punktów przez liczbę węzłów, na każdym węźle wylosowanie zadanej części punktów i zliczenie punktów w okręgu, a finalnie przesłanie wyników do jednego węzła. Jedyna część programu, która będzie musiała wykonać się sekwencyjnie, to zsumowanie wyników z poszczególnych węzłów i wyliczenie wartości π . Narzut na komunikację jest minimalny – każdy węzeł musi wysłać jedną liczbę, tj. liczbę wylosowanych punktów w okręgu.

Przygotowanie kodu

Wersja sekwencyjna

Stwórz sekwencyjną wersję programu wykorzystującego metodę MC do wyznaczania liczby π . Opis algorytmu znajdziesz we wprowadzeniu. Zapewnij możliwość podania liczby punktów jako argument wywołania programu.

Uruchom kod kilkakrotnie upewniając się, że za każdym razem zwracany wynik jest zbliżony do π (np. 2.8, 3.14, 4, ale nie 10 albo 0). Wartości poza zakresem $[0, 4]$ sugerują poważne błędy wynikające z implementacji lub niewłaściwych typów danych.

Opcjonalnie: uruchom program wielokrotnie dla różnych rozmiarów problemu (np. 1e2, 1e4, 1e6, 1e8), dla każdego wyliczając średni wynik i odchylenie standardowe. Jaki wpływ na te wartości ma rozmiar problemu?

Wskazówki:

- Wylosowanie liczb równomiernie rozłożonych na prostokącie jest równoważne z wylosowaniem obu współrzędnych niezależnie.
- W celu sprawdzenia czy punkt należy do okręgu możesz wykorzystać równanie okręgu.
- W celu obserwacji efektywności zrównoleglenia konieczne będzie losowanie dużych liczb punktów, np. 1e12. Upewnij się, że funkcja konwertująca parametr z konsoli na liczbę operuje na zmiennych odpowiedniego rozmiaru i nie doprowadzi do przepełnienia (to samo tyczy się używanych dalej we własnym kodzie typów zmiennych). `int` na pewno nie wystarczy, `long` wygląda lepiej, ale też ma swoje limity – warto zerknąć do specyfikacji dla pewności :)

Wersja równoległa

Zmodyfikuj program tak, żeby korzystając z MPI rozdzielał problem na wiele procesorów. Dodaj pomiar czasu w odpowiednich miejscach. Uważaj, żeby mierzyć tylko faktyczny czas pracy programu – dodaj bariery tam gdzie to konieczne.

Program nie wymaga wymiany informacji na początku obliczeń – nie będzie więc potrzebna komunikacja typu `broadcast` / `scatter`. Konieczne jest jednak zebranie wyników obliczeń każdego z procesorów w jednym i obliczenie π . Możesz skorzystać z operacji `gather`, jednak w tym przypadku lepszym wyjściem będzie operacja `reduce`. Ponieważ znana jest liczba wszystkich punktów, potrzebna jest tylko suma punktów wewnątrz okręgu we wszystkich losowaniach.

Uruchom kod kilkakrotnie upewniając się, że wynik nadal jest zbliżony do π , a czas maleje dla większej liczby procesorów (do liczby procesorów wspieranej przez maszynę, na której jest uruchamiany).

Wskazówki:

- <https://mpitutorial.com/tutorials/mpi-reduce-and-allreduce/>
- Pamiętaj, że funkcje implementujące komunikację MPI w C nie zwracają wyników – wyniki trafiają do zmiennych, których adresy są przekazywane jako parametry.
- W tym programie synchronizacja (przez bariery) potrzebna jest tylko na początku – na końcu synchronizacja jest wymuszona przez komunikację zbiorową.
- Postawienie bariery przed pomiarem czasu ma sens, gdyż zapewnia, że procesy są wolne i nie wykonują obliczeń zanim główny proces rozpocznie pomiar czasu. Postawienie bariery po starcie pomiaru czasu również ma sens, gdyż zapewnia poprawność programu współbieżnego — mamy pewność, że żaden proces nie zaczął wykonywać obliczenia przed pomiarem czasu. Warto zaznaczyć, że bariery te nie są częścią rozwiązywanego problemu, a tylko koniecznością wynikającą z potrzeby zmierzenia czasu.

Aby dostać punkty za aktywność na zajęciach należy zaimplementować wersję równoległą i pokazać, że działa.

Zadanie domowe

Przygotowanie

Upewnij się, że kod liczący π działa poprawnie – jeżeli zawierał inne elementy, np. wypisy częściowych czasów lub kilkakrotne powtórzenia i wyliczanie odchylenia standardowego, to na potrzeby tego zadania usuń te elementy. Kod powinien zawierać jedynie konieczne inicjalizacje/finalizacje MPI, obliczenia przybliżające π oraz pomiary.

Eksperymenty w środowisku vCluster

Uwaga: Nie należy wykonywać dużych pomiarów na środowisku vCluster. Służy ono tylko do testowania!

Zaloguj się do środowiska vCluster.

Przygotuj skrypt, który posłuży do powtarzalnego uruchamiania eksperymentów. Każdy eksperyment będzie wyglądać podobnie – uruchom kod dla 1–12 procesorów, zbierz pomierzone czasy wykonania. Za pomocą `machinefile` upewnij się, że z każdej maszyny wykorzystywany jest jeden procesor – inaczej wyniki będą zaburzone.

Przygotuj dwa warianty eksperymentu:

- Skalowanie silne – rozmiar problemu jest stały dla całego eksperymentu, efektywnie dzieląc się na coraz mniejsze części ze wzrostem liczby procesorów.
- Skalowanie słabe – rozmiar problemu jest stały dla każdego procesora, więc całkowity rozmiar problemu rośnie wraz ze wzrostem liczby

procesorów.

Wykonaj każdy z eksperymentów przynajmniej raz. Uzyskane czasy przedstaw na wykresie czasu od liczby procesorów. Wylicz i wykreśl wartości przyspieszenia na podstawie pomiarów czasu – pamiętaj, że wzór różni się nieco w obu wersjach eksperymentu.

Eksperymenty w środowisku Ares

Przygotuj skrypt (lub skrypty), który uruchomi program w obu wersjach skalowania. Uruchom eksperymenty dla zakresu 1–12 rdzeni, dla 3 różnych rozmiarów problemu:

- duży problem – taki, żeby wersja liczona na maksymalnej liczbie rdzeni (najszybsza) wykonywała się co najmniej minutę,
- mały problem – taki, żeby wersja liczona na jednym rdzeniu wykonywała się wyraźnie poniżej sekundy – im mniej, tym lepiej, ale na tyle, żeby pomiar nie zwrócił 0,
- średni problem – w przybliżeniu średnia geometryczna pomiędzy małym i dużym.

Każdy eksperyment powtórz kilkukrotnie (najlepiej ok. 10–30x), a czasy z powtórzeń uśrednij.

Każdą całą serię (1–12 rdzeni) danego eksperymentu można uruchomić w ramach jednego zadania – wiele uruchomień programu w ramach jednego skryptu. Podobnie można połączyć różne rozmiary i różne warianty. Może się to okazać szybsze, niż czekanie na dziesiątki zadań w kolejce – system kolejkowy przyznaje kary do priorytetu, jeżeli jest ich zbyt wiele. Ważne jest jednak, żeby kolejne powtórzenia do statystyki kolejkowość osobno – inaczej możemy nie wyeliminować np. problemów na jednym konkretnym węźle, który mieliśmy pecha otrzymać.

Podsumowując, powtórzenia powinny wyglądać następująco:

- dla każdego typu skalowania:
 - dla trzech różnych rozmiarów:
 - dla każdego powtórzenia:
 - dla liczby rdzeni 1–12:
 - uruchom kod i pomierz czas.

Wskazówki:

- Nie ma potrzeby przejmować się zaokrągleniem przy podziale punktów pomiędzy procesy. Dzielenie całkowitoliczbowe spowoduje, że w najgorszym przypadku zgubimy mniej niż jeden punkt na proces/rdzen, a więc dla N procesorów będzie to N-1 punktów. W porównaniu z liczbami punktów wymaganymi do wykonania zadania, jest to pomijalny szczegół.

Sprawozdanie

Wyniki tych eksperymentów stanowią drugą część sprawozdania z MPI, w połączeniu z pomiarami komunikacji P2P (zadanie 1). Za całość sprawozdania można zdobyć 11 punktów (6 za wiedzę i kody, 5 za wykresy).

Do zadania 1 zbieramy dane z vCluster. Do zadania 2 zbieramy dane z Aresa. W obu przypadkach należy testować program na vCluster aby upewnić się, że działa.

Dla obu typów skalowania, dla każdego rozmiaru eksperymentu należy przygotować dane do 4 wykresów metryk przedstawiających:

- zależność czasu od liczby procesorów (średnie wraz z odchyleniami standardowymi),
- zależność przyspieszenia (speedup) od liczby procesorów,
- zależność efektywności (efficiency) od liczby procesorów,
- zależność części sekwencyjnej (serial fraction) od liczby procesorów.

Sugestia: Wykorzystaj narzędzie, które pozwala na rysowanie wielu serii na jednym wykresie – połącz różne rozmiary tego samego eksperymentu w jeden wykres, dodając do niego przebieg idealny (dla skalowania liniowego). W efekcie mogą powstać 4 wykresy – jeden dla każdej metryki.

Każdy rozmiar powinien zostać uwzględniony w opisie dołączonym do wykresów – należy skomentować zachowanie każdej serii w kontekście teorii zawartej na wykładzie, a do tego porównać wszystkie serie jednego wykresu ze sobą i z wartościami idealnymi. Wyjaśnij też, czy wyliczasz przyspieszenie względne czy bezwzględne oraz jakie ma to konsekwencje w tym konkretnym problemie, a także w ogólnym przypadku.

Elementy konieczne do zawarcia w sprawozdaniu:

- Kod programów i skryptów (jako załącznik, nie w PDF).
- Dane pomiarowe (jako załącznik), komentarz odnośnie ich pozyskania (w PDF).
- Wykresy – jak w opisie wyżej.
- Opisy i wnioski do wykresów.

Wskazówki:

- Wykresy przyspieszenia powinny w idealnym przypadku mieć postać wykresu $y = x$ – jeżeli wyniki są istotnie różne od spodziewanych, spróbuj zwiększyć liczbę punktów. Czas obliczeń powinien wynosić co najmniej kilka sekund dla najmniejszego problemu, żeby np. zaburzenia komunikacji nie dominowały całego pomiaru.
- Linie łączące punkty na takim wykresie nie są wynikiem eksperymentu – nie istnieje czas wykonania dla 7.625 procesorów. Jeżeli zamieszczasz linie na wykresie, upewnij się, że nie są "na pierwszym planie" (np. są przerywane, przyszarzone, półprzezroczyste), a wykres

zawiera też wyraźne punkty faktycznych pomiarów.

- Wykresy powinny być czytelne bez dużego wysiłku – pamiętaj o tytule, podpisach osi, legendzie, jednostkach na osiach, odpowiednio dużym rozmiarze tekstu. Pamiętaj, że łączenie punktów na wykresie w łamaną ma sens tylko wizualny, dla zaobserwowania trendu – jeżeli łączysz punkty, upewnij się że linia nie przysłania faktycznych punktów.
- Poprawnie wykonane ćwiczenie powinno dać co najmniej dwie istotnie różne charakterystyki metryk dla każdego wariantu eksperymentu – opisz dlaczego widoczne jest akurat takie zachowanie i z czego ono wynika, a jeżeli nie zgadza się z idealnymi wartościami – dlaczego.
- Jeżeli nie jesteś w stanie uzyskać zadowalających wyników, a masz pewność, że rozmiary problemów są dobrze dobrane, opisz jakie wyniki były spodziewane i zaproponuj wyjaśnienie dlaczego nie udało się ich uzyskać.
- W komentarzach zawrzyj informacje o ilości komunikacji między procesami dla każdego rozmiaru problemu, a także jak duży to nakład czasowy w stosunku do samych obliczeń.

Ostatnia modyfikacja: poniedziałek, 16 marca 2026, 11:50



**AKADEMIA GÓRNICZO-HUTNICZA
IM. STANISŁAWA STASZICA
W KRAKOWIE**

Platforma obsługiwana przez:
Centrum e-Learningu i
Innowacyjnej Dydaktyki AGH
Centrum Rozwiązań
Informatycznych AGH

Pobierz aplikację
mobilną



Wybierz
język

